



Agent, MCP & Run

De un LLM a un tutor multiagente por voz

Guía técnica integral: arquitectura, componentes, decisiones, flujo, operación local, voz, nube, seguridad, pruebas y evolución.

Proyecto	Demo para Google I/O Extended Guatemala City
Versión documentada	MVP 0.1.0 - seis fases implementadas
Fecha	16/07/2026

“Los agentes deciden. MCP expone capacidades. El modelo genera.”

Contenido

1. Resumen ejecutivo y objetivo
2. Conceptos fundamentales
3. Arquitectura general
4. Flujo completo de aprendizaje
5. Agent App y sistema multiagente
6. Servidor MCP
7. RAG y contenido educativo
8. Persistencia y modelos de dominio
9. API, interfaz y trazabilidad
10. Voz con Gemini Live
11. Proveedores de modelos
12. Google Cloud Run y seguridad
13. Pruebas y calidad
14. Decisiones y trade-offs
15. Ejecución local y demo
16. Mapa del repositorio
17. Limitaciones y evolución
18. Glosario y referencias

Cómo usar esta guía

- **Para la charla:** lea primero las secciones 1, 3, 4, 14 y 15.
- **Para desarrollar:** use las secciones 5 a 13 como mapa del código.
- **Para desplegar:** revise las secciones 11 y 12 antes de ejecutar gcloud.
- **Para mantener:** consulte decisiones, pruebas, limitaciones y el mapa del repositorio.

1. Resumen ejecutivo y objetivo

Agent, MCP & Run es un asistente educativo de inteligencia artificial que conversa por texto y, opcionalmente, por voz. Su propósito no es solamente responder preguntas: diagnostica el nivel del estudiante, recupera contenido propio, adapta la explicación, plantea una evaluación, guarda el resultado y recomienda el siguiente paso. Todo el recorrido se hace visible mediante eventos de trazabilidad diseñados para una charla técnica.

Idea central

La demostración separa deliberadamente razonamiento, orquestación e infraestructura. El LLM es el motor probabilístico; los agentes tienen responsabilidades y toman decisiones; MCP estandariza herramientas y recursos deterministas.

Qué entrega el MVP

- Conversación por texto y evaluación corta.
- Orquestador y tres especialistas definidos con Google ADK 2.x.
- Servidor MCP remoto e independiente con seis herramientas y un recurso.
- RAG local, corpus de 13 temas y fuentes explícitas.
- Persistencia local JSON y adaptador Firestore.
- Interfaz moderna con progreso y panel de ejecución.
- Voz opcional con Gemini Live y fallback completo a texto.
- Preparación de dos servicios Cloud Run y proveedor opcional Foundry.
- 38 pruebas automatizadas sin credenciales cloud.

Qué no hace

- No trata al servidor MCP como agente.
- No persiste audio por defecto.
- No despliega recursos ni genera costos automáticamente.
- No expone chain-of-thought; sólo eventos y decisiones resumidas.
- No usa documentación extensa de terceros como contenido curricular.

2. Conceptos fundamentales

Concepto	Qué es	Cómo aparece en el proyecto
Modelo	Sistema probabilístico que transforma entradas en salidas.	Gemini, mock o Foundry detrás de ModelProvider.
Agente	Modelo + instrucciones + estado + herramientas orientados a un objetivo.	Root Orchestrator, Diagnostic, Tutor y Evaluator.
Subagente	Especialista con responsabilidad y capacidad de decisión propias.	Los tres especialistas declarados como sub_agents en ADK.
Herramienta	Operación acotada, validada y determinista.	Buscar contenido, leer progreso o guardar una evaluación.
MCP	Protocolo para descubrir herramientas, recursos y prompts.	Servicio remoto en /mcp/; no decide el flujo pedagógico.
RAG	Recuperar evidencia antes de generar una respuesta.	Índice local TF-IDF que devuelve fragmento y fuente.
Proveedor	Adaptador que encapsula una API de modelo.	GeminiModelProvider, MockModelProvider y FoundryModelProvider.
Repositorio	Interfaz para persistir dominio sin acoplar infraestructura.	JSON local o Firestore.

Regla para distinguir agente de herramienta

Si el componente debe interpretar un objetivo y elegir el siguiente paso, probablemente es un agente. Si recibe entradas validadas y ejecuta una operación definida, probablemente es una herramienta.

3. Arquitectura general

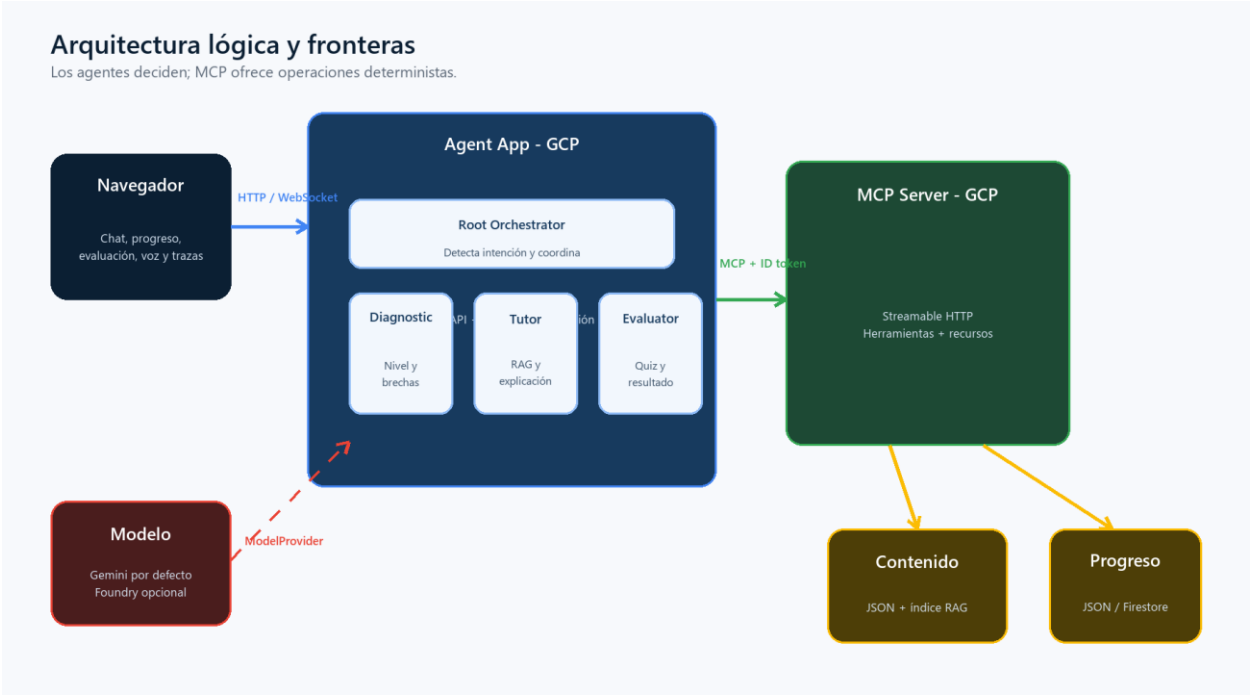


Figura 1. Componentes principales y límites de responsabilidad.

Lectura por capas

Capa	Responsabilidad	Implementación
Experiencia	Capturar texto/voz y mostrar respuesta, progreso, quiz y trazas.	HTML, CSS y JavaScript servidos por FastAPI.
Aplicación	Sesiones, API, correlation ID, errores y capacidades.	agent_app/api/main.py y servicios de sesión.
Agentes	Diagnosticar, enseñar, evaluar y coordinar.	Google ADK y clases de dominio en agent_app/agents.
Modelo	Generar explicaciones fundamentadas.	ModelProvider con Gemini, mock o Foundry.
Integración	Descubrir y llamar operaciones educativas.	Ciente MCP local o Streamable HTTP.
Dominio MCP	Contenido, recuperación, camino y progreso.	LearningService, Pydantic y repositorios.
Infraestructura	Escalar y proteger servicios independientes.	Cloud Run, Firestore, IAM y Artifact Registry.

Dos modos locales

- **Modo rápido:** MCP_USE_LOCAL_ADAPTER=true. El Agent App llama directamente a LearningService; ideal para plan B sin red.
- **Modo realista:** MCP_USE_LOCAL_ADAPTER=false. El Agent App se comunica con el segundo proceso mediante Streamable HTTP.

4. Flujo completo de aprendizaje

Secuencia de una sesión de aprendizaje

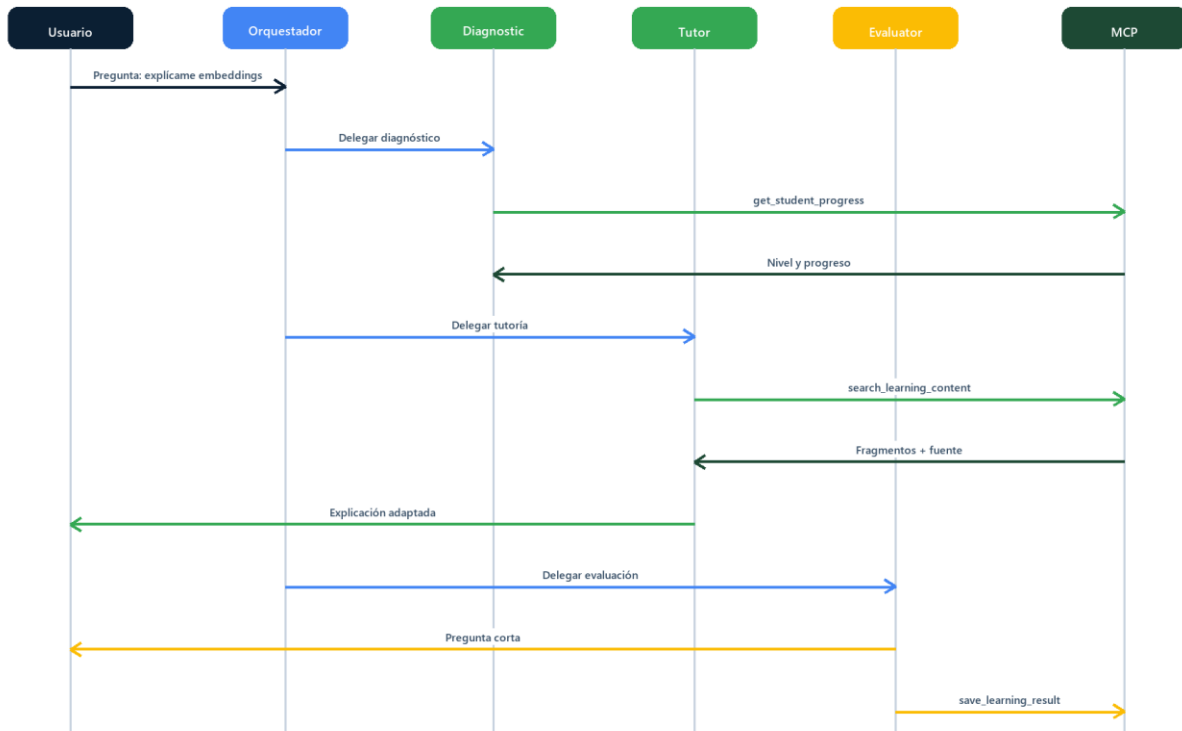


Figura 2. Secuencia desde la pregunta hasta el guardado del resultado.

1. El usuario envía una pregunta y un `student_id`.
2. El orquestador detecta que la intención es educativa e identifica el tema.
3. Diagnostic Agent consulta `get_student_progress` y establece nivel y brechas.
4. Tutor Agent llama `search_learning_content` con tema y nivel.
5. El recuperador devuelve fragmentos con fuente y relevancia.
6. El proveedor de modelo genera una explicación usando sólo esa evidencia.
7. Evaluator Agent crea una pregunta corta y la sesión guarda la evaluación pendiente.
8. El usuario responde; Evaluator calcula el puntaje y el feedback.
9. `save_learning_result` persiste el resultado y actualiza nivel y recomendaciones.
10. La interfaz muestra el resultado y la secuencia de agentes/herramientas.

Qué ve el público

Actor, acción, resumen, duración, éxito y resultado.
No ve prompts ocultos, tokens de razonamiento ni chain-of-thought.

5. Agent App y sistema multiagente

5.1 Root Orchestrator

Es la entrada lógica del sistema. Detecta el tema usando alias normalizados, crea o reutiliza una sesión, decide qué especialista interviene y agrega eventos de trazabilidad. No genera directamente la explicación curricular: delega a Tutor Agent después del diagnóstico.

- Recibe ChatRequest y produce ChatResponse.
- Genera correlation_id y session_id cuando no existen.
- Delega en orden diagnóstico, tutoría y evaluación.
- Mantiene la evaluación pendiente en InMemoryEvaluationStore.
- Convierte errores de tema o sesión en respuestas HTTP claras.

5.2 Diagnostic Agent

Consulta el progreso mediante la frontera LearningTools. Devuelve un Diagnostic estructurado con estudiante, tema, nivel, progreso y conocimientos faltantes. Si no hay evaluaciones, lo declara en lugar de asumir experiencia previa.

5.3 Tutor Agent

Recupera fragmentos, construye un prompt con pregunta, tema, nivel, fuente y evidencia, y solicita al ModelProvider una explicación. Su instrucción exige trabajar sólo con el material recuperado, adaptar la profundidad y reconocer evidencia insuficiente.

5.4 Evaluator Agent

Elige una pregunta breve por tema, compara palabras clave de forma determinista, calcula el porcentaje, produce feedback y solicita a MCP guardar el resultado. El mecanismo es simple a propósito: durante la charla se puede explicar y probar sin depender de una segunda evaluación por LLM.

5.5 Definición nativa ADK

agent_app/agent.py crea un root_agent de ADK con exactamente tres sub_agents. Diagnostic usa modo single_turn; Tutor y Evaluator usan modo task. Cada especialista recibe un McpToolset filtrado para exponer solamente las herramientas que necesita. Esto aplica mínimo privilegio también dentro del diseño del agente.

```
root_agent
├── diagnostic_agent [single_turn]
├── tutor_agent      [task]
└── evaluator_agent  [task]
```

6. Servidor MCP

El servidor usa FastMCP del SDK oficial, respuestas JSON y Streamable HTTP sin estado. Se monta en /mcp/ dentro de Starlette, con /healthz y /readyz. Sus funciones validan entradas con Pydantic y delegan en LearningService; no contienen comportamiento de agente.

Herramienta	Entrada	Salida y uso
get_student_progress	student_id	Nivel, temas, evaluaciones y recomendaciones.
search_learning_content	topic, level, limit	Fragmentos relevantes con fuente y score.
get_learning_path	student_id	Completados, próximos temas y porcentaje.
save_learning_result	student_id, topic, score, feedback, recommendation	Persistencia y progreso actualizado.
find_practical_example	topic, programming_language	Código breve y explicación.
list_available_topics	sin parámetros	Catálogo y niveles disponibles.
Recurso learning://topics	lectura MCP	Catálogo curricular como recurso JSON.

Por qué Streamable HTTP sin estado

- Es el transporte recomendado para MCP remoto.
- Permite ejecutar el servidor como servicio Cloud Run independiente.
- Evita mantener sesiones de progreso en memoria de una instancia.
- Facilita escalado horizontal porque la persistencia vive en JSON local de desarrollo o Firestore.

7. RAG y contenido educativo

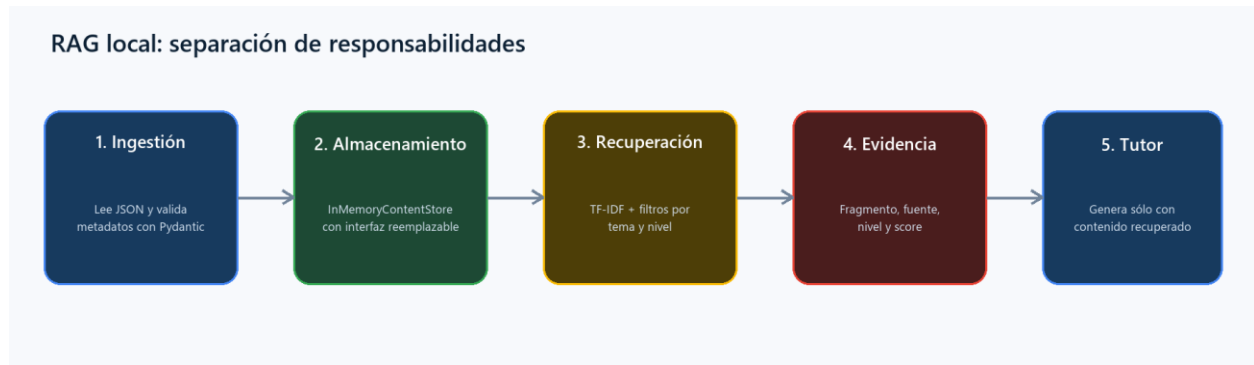


Figura 3. Pipeline local de ingestión, almacenamiento, recuperación y uso por Tutor Agent.

Contenido inicial

learning_content.json contiene material breve y propio sobre inteligencia artificial, Machine Learning, NLP, modelos de lenguaje, LLM, tokens, embeddings, ventana de contexto, RAG, tool calling, agentes, MCP y sistemas multiagente. Cada registro tiene id, tema, título, nivel, texto, fuente y palabras clave.

Cómo funciona la búsqueda

1. Se normaliza la consulta a minúsculas, ASCII y tokens alfanuméricos.
2. Se filtra por tema y se prioriza el nivel solicitado.
3. Si ese nivel no existe, se usa contenido del mismo tema como fallback.
4. Se calcula una puntuación TF-IDF por coincidencias de consulta y documento.
5. Se agrega un peso por coincidencia de tema y nivel.
6. Se ordenan los resultados y se devuelven fragmento, fuente y score.

Por qué no usamos embeddings en el primer índice

El objetivo de la demo es enseñar el patrón RAG, no esconderlo detrás de infraestructura. TF-IDF es determinista, económico, rápido y funciona sin credenciales. La interfaz ContentStore permite cambiar más adelante a búsqueda vectorial sin modificar agentes ni herramientas MCP.

Limitaciones del RAG local

- La recuperación léxica no captura todos los sinónimos.
- El corpus está en memoria y no está diseñado para gran volumen.
- No existe todavía un pipeline incremental desde Cloud Storage.
- La evaluación de calidad se basa en pruebas funcionales, no en métricas de recall/faithfulness a escala.

8. Persistencia y modelos de dominio

8.1 Modelos Pydantic

Modelo	Responsabilidad
LearningContent	Fragmento curricular validado con metadatos.
SearchResult	Evidencia recuperada con fuente y score.
Assessment	Tema, puntaje, feedback, recomendación y fecha.
StudentProgress	Nivel, temas, evaluaciones y recomendaciones del estudiante.
LearningPath	Avance y próximos conceptos.
ChatRequest / ChatResponse	Contrato de la conversación por texto.
EvaluationRequest / EvaluationResponse	Contrato de la evaluación.
TraceEvent	Evento observable sin razonamiento interno.

8.2 Repositorio JSON local

LocalProgressRepository usa un bloqueo reentrante, valida el archivo completo al leer y escribe en un archivo temporal antes de `os.replace`. El reemplazo atómico reduce el riesgo de dejar un JSON parcial si el proceso se interrumpe.

8.3 Adaptador Firestore

FirestoreProgressRepository implementa el mismo puerto. Lee un documento por estudiante y guarda la evaluación en una transacción. El agente no conoce Firestore: sólo llama una herramienta MCP, y MCP elige el adaptador según `MCP_PROGRESS_BACKEND`.

Cálculo del nivel

- Promedio reciente menor de 60: beginner.
- Promedio entre 60 y 84.99: intermediate.
- Promedio de 85 o más: advanced.
- Se consideran las últimas cinco evaluaciones para reflejar progreso reciente.

9. API, interfaz y trazabilidad

Endpoints principales

Ruta	Método	Propósito
/	GET	Interfaz web.
/healthz	GET	Salud del Agent App.
/api/capabilities	GET	Disponibilidad de texto y voz.
/api/chat	POST	Ejecutar diagnóstico, tutoría y crear quiz.
/api/evaluate	POST	Calificar y guardar resultado.
/ws/live	WebSocket	Puente seguro de audio con Gemini Live.

Correlation ID y logging

Un middleware acepta X-Correlation-ID o genera un UUID. El valor se devuelve en la respuesta y se incluye en logs JSON estructurados. Esto permite relacionar el comportamiento del API con los eventos que ve la interfaz.

Panel de trazabilidad

- Mensaje del usuario.
- Detección resumida de intención y tema.
- Delegaciones a cada especialista.
- Llamadas MCP y resultado resumido.
- Llamada al proveedor de modelo.
- Duración en milisegundos y estado de éxito.
- Respuesta final y guardado de progreso.

Decisión de seguridad y pedagogía

La trazabilidad enseña la arquitectura sin revelar razonamiento privado del modelo. Un evento dice 'tema detectado: embeddings', no muestra cadenas internas ni prompts ocultos.

10. Voz con Gemini Live

La voz es opt-in y se activa únicamente cuando `MODEL_PROVIDER=gemini` y existen credenciales válidas. El navegador nunca recibe la API key. Se conecta por WebSocket al backend, que abre la sesión Live API.

Flujo de audio

1. El navegador solicita permiso de micrófono sólo al pulsar el botón.
2. Web Audio captura mono y reduce la señal a PCM de 16 bits y 16 kHz.
3. Los bytes viajan al backend por WebSocket.
4. GeminiLiveBridge envía `audio/pcm;rate=16000` al SDK oficial.
5. Gemini devuelve audio PCM de 24 kHz y transcripciones.
6. El navegador agenda los buffers para reproducirlos sin cortes.
7. Si hay error, permiso denegado o desconexión, se conserva el chat de texto.

Privacidad y resiliencia

- No se escribe audio en disco ni base de datos.
- No se registran bytes de audio ni credenciales.
- La voz no bloquea health checks, chat ni evaluación.
- Con Foundry seleccionado, la voz se deshabilita para evitar mezclar proveedores de forma implícita.

11. Proveedores de modelos

ModelProvider define una sola operación asíncrona: generate(ModelRequest). La fábrica central decide qué implementación crear. Ningún agente contiene condicionales de Gemini o Azure.

Proveedor	Uso	Autenticación	Observaciones
Mock	Pruebas, offline y plan B.	Ninguna.	Respuesta determinista basada en evidencia.
Gemini	Proveedor predeterminado.	API key o identidad de Vertex AI.	Google Gen AI SDK y timeout.
Foundry	Alternativa multi-cloud.	DefaultAzureCredential / Entra ID.	Project endpoint + /openai/v1/responses.

Por qué la abstracción importa

- Evita dispersar if MODEL_PROVIDER por toda la aplicación.
- Permite probar los agentes sin red ni credenciales.
- Hace explícito cuándo los datos salen de GCP hacia Azure.
- Reduce el costo de sustituir SDKs o endpoints en el futuro.

Topología Foundry

Cuando se selecciona Foundry, la app, ADK, MCP, contenido y Firestore siguen en GCP. Sólo la generación del Tutor Agent cruza a Azure. El adaptador usa Responses API estable y no usa azure-ai-inference beta, APIs classic ni un agente hospedado en Foundry.

12. Google Cloud Run y seguridad

12.1 Dos servicios

- **learning-agent:** público, contiene UI, FastAPI, agentes, proveedores y WebSocket de voz.
- **learning-mcp:** privado, contiene herramientas, recuperación y acceso a Firestore.

12.2 Autenticación servicio a servicio

1. learning-agent corre con una cuenta de servicio administrada por el usuario.
2. Esa identidad recibe roles/run.invoker únicamente sobre learning-mcp.
3. Antes de la llamada MCP, el cliente obtiene un ID token firmado por Google.
4. La audiencia del token es la URL base del servicio MCP.
5. El token se envía como Authorization: Bearer sin registrarse.

12.3 Mínimo privilegio

Identidad	Rol	Motivo
learning-mcp	roles/datastore.user	Leer y escribir progreso en Firestore.
learning-agent	roles/aipatform.user	Consumir Gemini mediante Vertex AI.
learning-agent	roles/run.invoker sobre MCP	Invocar sólo el servicio privado.
learning-agent opcional	roles/secretmanager.secretAccessor	Leer únicamente secretos configurados.

12.4 Contenedores y health checks

Cada servicio tiene su Dockerfile basado en python:3.13-slim, instala sólo los grupos necesarios y cambia a un usuario sin privilegios. Los manifiestos incluyen startupProbe y livenessProbe contra /healthz, límites de CPU/memoria y escalado de cero a diez instancias.

12.5 Secretos y costos

- Vertex AI usa identidad de servicio y evita API key en el despliegue predeterminado.
- Gemini Developer API y credenciales Entra deben llegar desde Secret Manager.
- El repositorio no despliega ni destruye recursos automáticamente.
- deploy.sh debe ejecutarse sólo después de revisar cuotas, región, IAM y presupuesto.
- Rollback se realiza enviando tráfico a una revisión anterior de Cloud Run.

13. Pruebas y calidad

La suite final contiene 38 pruebas. Las pruebas normales no llaman servicios cloud ni requieren credenciales. Se usan repositorios temporales, clientes simulados y un servidor MCP local real para verificar el transporte.

Área	Qué se prueba
Contenido	Cobertura de 13 temas, validación y ids únicos.
Recuperación	Nivel, fallback, fuente, score y tema desconocido.
Persistencia	JSON atómico, corrupción explícita, nivel y Firestore simulado.
MCP	Health, initialize, tools/list, resources/list, tools/call y entrada inválida.
Routing	Detección de tema y rechazo de mensajes ambiguos.
Delegación	Participación de los tres especialistas y eventos esperados.
API	Chat completo, evaluación, static assets, capabilities y errores.
Integración	Agent App llama un MCP independiente por Streamable HTTP real.
Proveedores	Selección, credenciales, Gemini, Foundry y Responses API simulada.
Voz	Habilitación/deshabilitación sin exponer la clave.

Validaciones adicionales realizadas

- compileall para sintaxis Python.
- pip check sin dependencias rotas.
- docker compose config válido.
- YAML de Cloud Build y Cloud Run parseados.
- node --check sobre app.js.
- Smoke test HTTP real con health, capabilities y chat de embeddings.
- Escaneo sin valores con patrón de credencial.

Advertencias conocidas

ADK 2.5 emite advertencias internas sobre BaseAgentConfig y autenticación experimental. No producen fallos. La voz, Firestore, Cloud Run y Foundry reales no se probaron porque requieren credenciales o pueden generar costos.

14. Decisiones y trade-offs

Decisión	Por qué	Costo o riesgo
MCP 1.x con límite <2	La línea 1.x era estable y 2.x estaba en alfa.	Habría que planificar migración cuando 2.x sea estable.
Streamable HTTP stateless	Escala bien en Cloud Run y es el transporte remoto recomendado.	Cada llamada puede abrir sesión; conviene optimizar conexiones a escala.
RAG léxico local	Cero credenciales, determinista y explicable en una charla.	Menor calidad semántica que embeddings/vector search.
Tres especialistas	Cada uno tiene una responsabilidad genuinamente distinta.	Evita proliferar agentes para operaciones que son herramientas.
Evaluación determinista	Fácil de probar y mostrar; no duplica costo de LLM.	La calidad pedagógica es limitada y debe evolucionar.
MCP separado	Demuestra la frontera real, permite seguridad y despliegue independiente.	Aumenta latencia y operación frente a una función local.
ModelProvider	Aísla APIs y habilita mock/Gemini/Foundry.	No todos los proveedores soportan idénticas capacidades.
JSON + Firestore	Desarrollo simple y producción durable con el mismo puerto.	El adaptador local no soporta múltiples procesos concurrentes.
Voz posterior al texto	La demo nunca depende de micrófono, red o modelo Live.	La voz no completa por sí sola el flujo de evaluación persistida.
Foundry sólo como modelo	Mantiene arquitectura y agentes en GCP.	Los prompts cruzan nubes y requieren gobernanza adicional.

15. Ejecución local y demo

15.1 Instalar

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev,agents,foundry]"
```

15.2 Modo rápido sin red

```
$env:MODEL_PROVIDER="mock"
$env:MCP_USE_LOCAL_ADAPTER="true"
python -m agent_app.api.main
```

15.3 Dos procesos con MCP real

```
# Terminal 1
python -m mcp_learning_server.server

# Terminal 2
$env:MODEL_PROVIDER="mock"
$env:MCP_USE_LOCAL_ADAPTER="false"
$env:MCP_SERVER_URL="http://localhost:8001/mcp/"
python -m agent_app.api.main
```

15.4 Docker Compose

```
docker compose up --build
```

15.5 Pruebas

```
python -m pytest
```

15.6 Conversación recomendada

	Pregunta Explícame qué son los embeddings, pero primero comprueba cuánto sé.
	Respuesta de evaluación Un embedding es un vector de significado y la similitud permite comparar elementos cercanos.

Plan B de la charla

- Sin modelo: MODEL_PROVIDER=mock.
- Sin red: MCP_USE_LOCAL_ADAPTER=true.
- Sin voz: usar texto; el botón queda deshabilitado.
- Sin Cloud Run: mostrar arquitectura, manifiestos y health checks locales.
- Si MCP falla: reiniciar sólo el segundo proceso.

16. Mapa del repositorio

agent_app/	
agents/	Orquestador y tres especialistas
api/	FastAPI, endpoints y middleware
models/	Contratos HTTP y trazabilidad
providers/	Mock, Gemini y Foundry
services/	Cliente MCP, voz, logging y sesiones
static/	HTML, CSS y JavaScript
mcp_learning_server/	
content/	Corpus educativo propio
repositories/	JSON local y Firestore
services/	Ingestión, almacenamiento, retrieval y casos de uso
tools/	Adaptadores de herramientas MCP
tests/	
unit/	Pruebas aisladas
integration/	API y transporte MCP real
infra/cloudrun/	Docker, Cloud Build y manifiestos
docs/	Arquitectura, demo, deployment y Foundry

Archivo clave	Qué buscar allí
agent_app/agent.py	Jerarquía ADK y McpToolset por especialista.
agents/orchestrator.py	Routing, delegación y trazas.
api/main.py	Composición de dependencias y endpoints.
services/learning_tools.py	Adaptador local, MCP remoto e ID token.
services/live_voice.py	Puente WebSocket a Gemini Live.
mcp_learning_server/server.py	FastMCP, health checks y repositorio elegido.
services/retrieval.py	Tokenización y TF-IDF.
tools/learning_tools.py	Contratos MCP validados.
repositories/local_progress.py	Persistencia JSON atómica.
infra/cloudrun/deploy.sh	Secuencia reproducible de despliegue.

17. Limitaciones y evolución

Limitaciones actuales

- El índice local no es semántico ni está pensado para grandes volúmenes.
- El evaluador usa palabras clave y no mide comprensión profunda.
- InMemoryEvaluationStore se pierde al reiniciar y no escala entre instancias.
- La voz conversa, pero la evaluación persistida se completa por texto.
- No se verificaron servicios cloud reales ni costos/cuotas.
- La interfaz no tuvo inspección visual automatizada por una restricción del runtime de navegador; sí se verificaron sus recursos y flujo HTTP.

Siguientes mejoras recomendadas

1. Persistir sesiones y evaluaciones pendientes en Firestore o Redis.
2. Agregar embeddings y un ContentStore vectorial con evaluación de retrieval.
3. Implementar ingestión desde Cloud Storage con versionado de corpus.
4. Mejorar el evaluador con rúbricas estructuradas y respuestas esperadas por nivel.
5. Reutilizar conexiones MCP y añadir circuit breaker si aumenta el tráfico.
6. Añadir métricas OpenTelemetry exportables y dashboards.
7. Probar Live API, Firestore, Cloud Run y Foundry en entornos controlados con presupuesto.
8. Crear pruebas end-to-end del navegador y accesibilidad.
9. Evaluar migración a MCP 2.x sólo cuando la línea sea estable.

18. Glosario y referencias

Término	Definición breve
ADK	Agent Development Kit de Google para construir y ejecutar agentes.
ASGI	Interfaz asíncrona usada por FastAPI, Starlette y Uvicorn.
Correlation ID	Identificador que relaciona logs y eventos de una solicitud.
Embedding	Vector numérico que representa características o significado.
Grounding	Vincular la respuesta del modelo con evidencia recuperada.
ID token	Token OIDC que prueba la identidad del servicio llamador.
MCP	Model Context Protocol para herramientas, recursos y prompts.
RAG	Retrieval-Augmented Generation: recuperar antes de generar.
Streamable HTTP	Transporte MCP remoto que admite JSON y streaming.
TF-IDF	Puntuación léxica basada en frecuencia local e inversa del corpus.

Documentación interna

- README.md - ejecución y capacidades.
- docs/architecture.md - componentes, secuencia y trade-offs.

- docs/demo-script.md - guion y plan B.
- docs/deployment.md - Cloud Run, IAM, verificación y rollback.
- docs/foundry.md - topología multi-cloud y Responses API.
- docs/phase-1.md - base local y recuperación.

Referencias oficiales consultadas

Fuente	URL
Google ADK 2.0	https://adk.dev/2.0/
ADK collaborative workflows	https://adk.dev/workflows/collaboration/
ADK MCP tools	https://adk.dev/tools-custom/mcp-tools/
MCP Python SDK	https://github.com/modelcontextprotocol/python-sdk
Gemini Live API	https://ai.google.dev/gemini-api/docs/live-api/get-started-sdk
Cloud Run service-to-service auth	https://cloud.google.com/run/docs/authenticating/service-to-service
Cloud Run health checks	https://cloud.google.com/run/docs/configuring/healthchecks
Microsoft Foundry Responses API	https://learn.microsoft.com/azure/foundry/agents/quickstarts/responses-api

Cierre

El valor del proyecto no está en usar muchas tecnologías, sino en mostrar fronteras claras: el modelo genera, cada agente tiene una responsabilidad, MCP estandariza capacidades, el contenido fundamenta la respuesta y la infraestructura puede cambiar sin reescribir el dominio.